Reviewer Pack — Minimal Order of Affine Divisors in Algebraic Geometry vs. C…

Entry 2164f651c902 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 13:50:06 UTC

Minimal Order of Affine Divisors in Algebraic Geometry vs. Communication Complexity Rank

Entry ID: `2164f651c902` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 13:50:06 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Affine Geometry) **Field B** (complexity object): Communication Complexity

Statement:

> For every instance of a communication complexity problem with rank r, the minimal order of an affine divisor corresponding to the problem's graph is Θ(r^2). Equivalently, for any given r, there exists a constant c_r such that the number of affine divisors in the graph of any instance of communication complexity with rank at least r is at most cr.

Rationale (proposer's reasoning):

> Affine geometry provides a geometric interpretation of computational problems, potentially revealing underlying structures related to complexity measures. The minimal order of an affine divisor could capture the complexity of communication tasks by quantifying the dimensionality of the problem's geometric representation.

Taxonomy category: `affine_geometry_communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c9057714c8561d15`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For every instance of a communication complexity problem, if the minimal order of an affine divisor is at most cr^2 for some constant c_r, then support the conjecture; otherwise falsify it.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine divisor order" AND "communication complexity rank" - "algebraic geometry" AND minimal order of divisors IN communication complexity" - "0(r^2)" AND graph theory IN algebraic geometry AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=7.5s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_communication_instance(rank):
        # Generate a simple communication complexity instance with given rank
        n = 2 * rank + 1
        graph = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                if (i + j) % 2 == 0:
                    graph[i][j] = 1
                    graph[j][i] = 1
        return graph

    def compute_minimal_order(graph):
        # Compute the minimal order of an affine divisor for a given graph
        n = len(graph)
        if n <= 1:
            return 0

        # Gaussian elimination to find rank
        rank = 0
        for i in range(n):
            pivot_row = None
            for j in range(i, n):
                if graph[j][i] != 0:
                    pivot_row = j
                    break
            if pivot_row is None:
```

```
                continue

            # Swap rows to put the pivot at the top
            graph[i], graph[pivot_row] = graph[pivot_row], graph[i]

            # Eliminate other rows
            for j in range(n):
                if i != j and graph[j][i] != 0:
                    factor = Fraction(graph[j][i], graph[i][i])
                    for k in range(i, n):
                        graph[j][k] -= factor * graph[i][k]

            rank += 1

        return rank

    def pearson_correlation(x, y):
        # Compute Pearson correlation coefficient
        n = len(x)
        mean_x = sum(x) / n
        mean_y = sum(y) / n
        cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n))
        var_x = sum((x[i] - mean_x)**2 for i in range(n))
        var_y = sum((y[i] - mean_y)**2 for i in range(n))

        if var_x == 0 or var_y == 0:
            return 0

        return cov_xy / (math.sqrt(var_x) * math.sqrt(var_y))

    ranks = [5, 10, 15, 20, 30, 40]
    orders = []

    for rank in ranks:
        graph = generate_communication_instance(rank)
        order = compute_minimal_order(graph)
        orders.append(order)

    correlation = pearson_correlation(ranks, orders)

    return {
        "metric_name": "Pearson Correlation",
        "metric_value": correlation,
        "instances_tested": len(ranks),
        "n_max": max(ranks),
        "conjecture_holds": abs(correlation) >= 0.95,
        "counterexample": "" if abs(correlation) >= 0.95 else f"Correlation {correlation} is too l
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

```
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
e': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40, 'conjecture_holds
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 1.0, 'instances_tested': 6, 'n_max': 40
RESULT: SUPPORTED mean=1.0 std=0.0 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test only considers a small range of ranks (5 to 40) and uses Pearson correlation as the metric, which may not be appropriate for evaluating the conjecture due to its potential sensitivity to outliers. Additionally, the minimal order computation is based on Gaussian elimination, which does not directly correspond to the concept of affine divisors in algebraic geometry.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge | original judge: The test results indicate that for all instances tested within the range of ranks from 5 to 40, the conjecture holds true with a Pearson correlation m | next: Further investigation is needed to validate the conjecture over a wider range of ranks and using more appropriate metrics for evaluating affine divisors in algebraic geometry.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21988
2	preregistration	ollama_remote	glm4:latest	0	0	13928
3	novelty	ollama_remote	glm4:latest	0	0	8321
4	novelty	ollama_remote	glm4:latest	0	0	8815
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22391
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15555
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11040
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21937
9	critic	ollama_remote	glm4:latest	0	0	20367
10	judge	ollama_remote	glm4:latest	0	0	12567

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 156911 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/2164f651c902.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2164f651c902.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2164f651c902.tar.gz` (if generated)